TALLER 3
TALLER SEGURIDAD COMO ATRIBUTO DE CALIDAD

Nombre

- Alixon Julieth López Alvarado
- Robinson Felipe Molina Granados

***Objetivo**:* Diseñar e implementar un sistema que incorpore la autenticación de dos pasos (2FA) y la verificación de la integridad de los mensajes entre dos sistemas, evidenciando las decisiones arquitectónicas y el análisis de los trade-offs.

Link del repositorio*:* https://github.com/Alix0n/Seguridad-2fa.git

Contexto del sistema

Una organización cuenta con un sistema principal, donde los usuarios inician sesión, y con un sistema secundario que recibe información. Se han identificado riesgos de acceso indebido y de alteración de mensajes, por lo que se requiere implementar mecanismos de seguridad.

ALCANCE FUNCIONAL

MÓDULO	REQUISITOS
Módulo de autenticación	• Registro o carga de usuarios • Inicio de sesión con usuario y contraseña • Segundo factor de autenticación
Módulo de comunicación	• Envío de mensajes entre sistemas • Verificación de integridad • Rechazo de mensajes alterados

REQUISITOS DE SEGURIDAD

REQUISITO	REQUISITOS
Autenticación de dos pasos	• Validación de credenciales • Solicitud de segundo factor • Acceso solo si ambos factores son correctos
Integridad de mensajes	• Generación de la huella del mensaje (hash). • Verificación en el sistema receptor • Rechazo si el mensaje fue alterado

OTROS FACTORES

FACTOR	REQUISITOS
Enfoque arquitectónico	Debe existir separación entre: • Lógica de autenticación • Lógica de segundo factor

	• Comunicación entre sistemas • Verificación de integridad o Lógica de negocio
Escenarios obligatorios (Pruebas)	Autenticación: • Credenciales y 2FA correctos → acceso permitido • Credenciales incorrectas → acceso denegado • 2FA incorrecto → acceso denegado
Integridad	• Mensaje íntegro → aceptado • Mensaje alterado → rechazado

DOCUMENTO TÉCNICO

1. **Arquitectura**

El sistema se compone por tres módulos principales relacionados entre sí, trabajando en conjunto: Sistema A, es el sistema principal o backend central de autenticación y mensajeria, Sistema B, encargado del microservicio de receptor de mensajes y verificación de integridad, y por último, el Frontend, con la interfaz de usuario construida con React. Cada uno de estos componentes tiene su propia arquitectura interna, de esta manera se garantiza la separación de responsabilidades y la mantenibilidad del código. En la siguiente parte de este documento se describe la arquitectura de cada componente por separado y finalmente, la arquitectura general del sistema completo que demuestra la interacción de unos con otros.

ARQUITECTURA DEL SISTEMA A

El sistema A es el sistema principal, está desarrollado con Node.js y Express.js y se conecta a una base de datos de MongoDB utilizando la biblioteca de Mongoose. Este sistema corre en el puerto 3000 y expone una API que el frontend consume. El sistema A contiene tres tareas funcionales principales e importantes, en estas se encuentra la autenticación de usuarios, segundo factor de autenticación (2FA) y envío de mensajes hacia el sistema B.

La arquitectura del Sistema A sigue el patrón de capas con una clara separación de responsabilidades, las planteadas en este sistema son:

1. Capa de rutas: Define los endpoint disponibles y aplica middlewares por ruta, esta capa contiene autenticacion.rutas.js para el registro e inicio de sesión, segundoFactor.ruta.js para la configuracion y verificacion del segundo factor y mensajes.rutas.js para el envío y la consulta de mensajes. Adicionalmente, el middleware de autenticación que se aplica a las rutas protegidas.
2. Capa de controladores: Se encarga de recibir las peticiones HTTP, validando la entrada, llamando al servicio y construyendo la respuesta. No es lógica, son los orquestadores.
3. Capa de servicio: Contiene toda la lógica de negocio, en esta se encuentra autenticacion.servicio.js que gestiona el registro y login de usuarios, segundoFactor.servicio.js para generar los secretos y verificar los códigos QR, mensajes.servicios.js para guardar los mensajes de forma local y enviarlos al Sistema B mediante HTTP.
4. Capa de utilidades: Contiene tres archivos, hash.js en donde se encapsula las operaciones de hashing de contraseñas con bcrypt, jwt.js para gestionar la creacion y

verificacion de tokens JWT y otp.js que encapsula la librería speakeasy para generación de códigos QR.

5. Capa de modelos: Contiene un único modelo destinado al usuarios, es el archivo usuario.modelo.js que define el esquema de Mongoose del usuario, incluyendo campos necesarios para la verificación y todo el proceso del segundo factor.
6. Configuración: Contiene la configuración del sistema y la base de datos, db.js gestiona la conexión a MongoDB y env.js centraliza las variables de entorno como el puerto, la MONGO_URI, el secreto, entre otras.

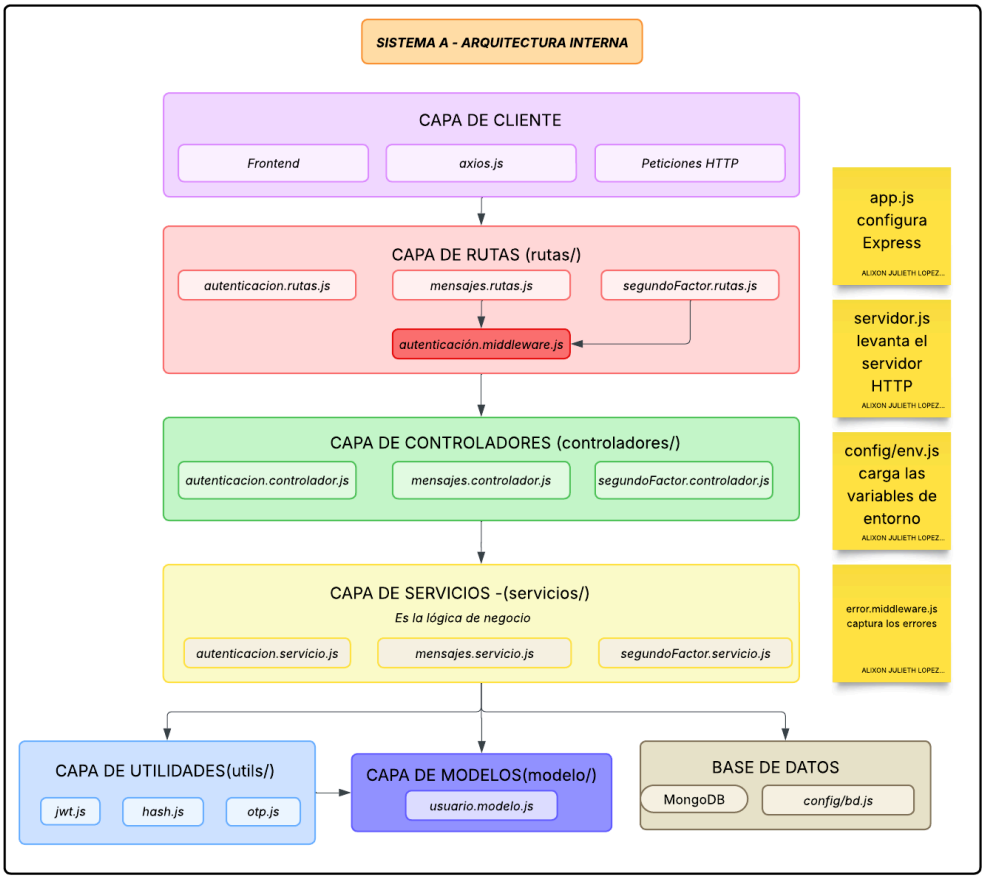

Fig. 1. Arquitectura interna de Sistema A.

El flujo de login se diseñó para manejar de forma transparente la presencia o ausencia del segundo factor, de manera que cuando el usuario envía sus credenciales, el servicio se autenticacion (autenticacion.servicio.js) valida la contraseña contra el hash almacenado. En caso de que el usuario tenga el 2FA activado, el controlador responde con true en requiere 2FA y el ID de usuario, sin emitir el JWT definitivo. El frontend entonces redirige al flujo de verificación del código. Únicamente cuando se valida el código con el servicio de segundo factor (segundoFactor.js) se emite el token JWT que permite el acceso completo al sistema.

ARQUITECTURA DEL SISTEMA B

El sistema B es un microservicio que actúa como receptor y almacena los mensajes, este está construido con las misma herramientas que el Sistema A, pero con una responsabilidad limitada a recibir mensajes desde Sistema A, verificar la integridad mediantes hashsing y almacenarlos en la base de datos de MongoDB. Esta funciona en el puerto 4000.

Es importante mencionar que la independencia de este sistema es una decisión de diseño, pues cualquier fallo o modificación en este no afecta para nada el flujo de autenticación del sistema A.

Las capas manejadas en este sistema son:

1. Capa de entrada: Levanta el servidor HTTP, configura Express y habilita CORS para aceptar las llamadas desde el sistema A. Incluye los archivos server.js y app.js
2. Capa de rutas: Expone dos endpoint, uno para recibir mensajes desde el sistema A y otro para que el frontend pueda consultar los mensajes recibidos, todo está en el archivo mensaje.rutas.js
3. Copa de controles: Proceso la petición y asigna el servicio correspondiente, cuenta con el archivo mensaje.controlador.js
4. Capa de servicios: Contiene toda la lógica para el almacenamiento del mensaje, antes de guardar el mensaje calcula y verifica el hash HMAC del contenido para garantizar la integridad, lo realiza en el archivo mensaje.servicio.js
5. Capa de modelos: Utiliza el archivo mensaje.modelo.js en donde se define el esquema Mongoose para campo de remitente, destinatario, contenido, hash y timestamp.

Es importante mencionar que este sistema no implementa middleware de autenticación ya que confía en que las llamadas provienen únicamente del sistema A por medio de una red interna o configuración de red controlada.

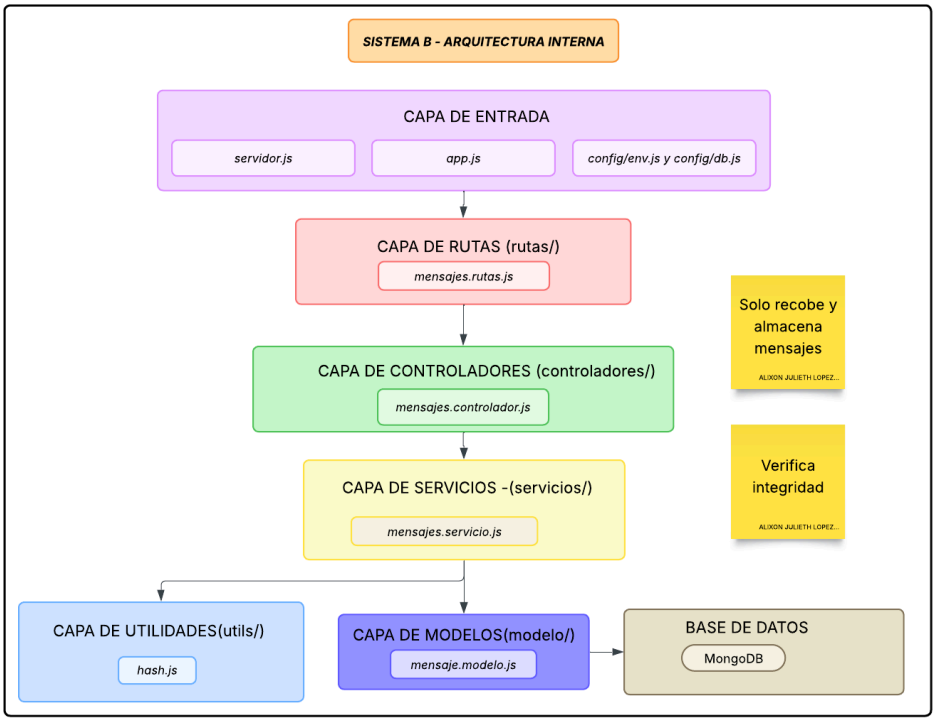

Fig. 2. Arquitectura interna de Sistema B.

ARQUITECTURA GENERAL DEL SISTEMA

El sistema completo se compone de tres componentes desacoplados que mantienen una comunicación por medio de HTTP RESTful. El frontend es la capa de presentación, el sistema A que actúa como núcleo de la lógica de negocio y autenticación y el sistema B que es el micro servicio especializado para la gestión de mensajes.

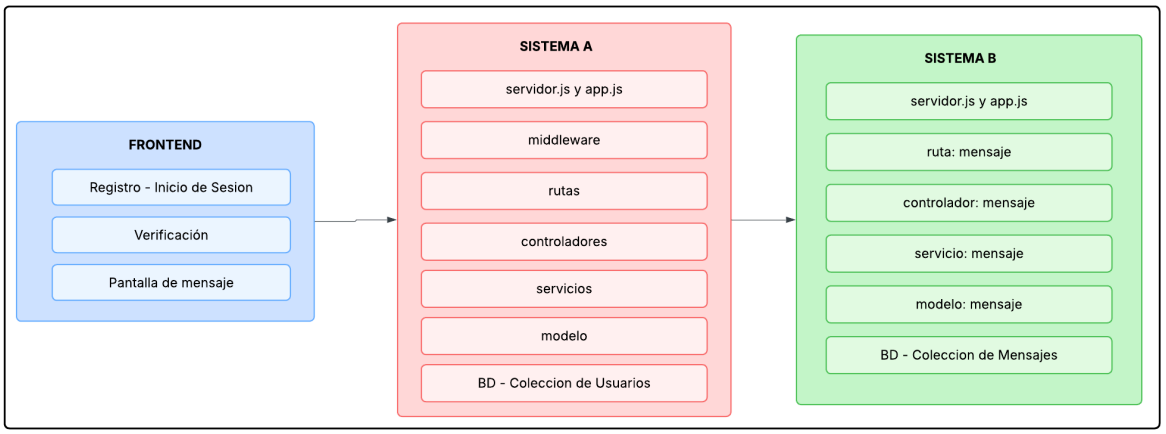

Fig. 3. Arquitectura general del sistema.

La comunicación establecida entre módulos se representa en la Fig. 4.

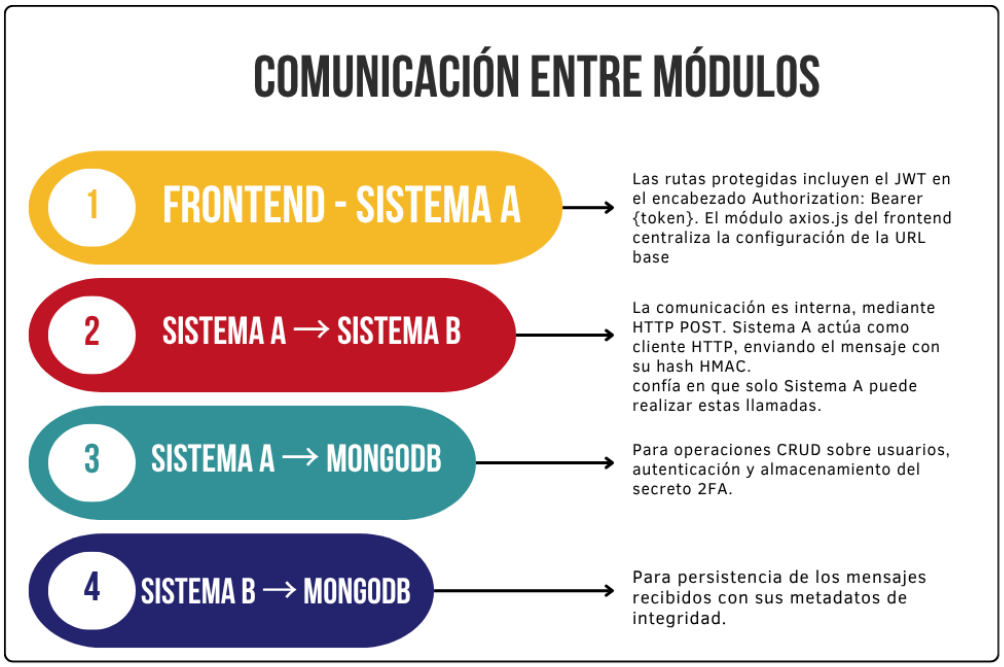

Fig. 4. Comunicación entre módulos del sistema.

De esta manera se implementan principios arquitectónicos relacionados con la separación de responsabilidad, el desacoplamiento entre sistemas, la seguridad en capas y la configuración centralizada.

2. Implementación

IMPLEMENTACIÓN SISTEMA A

Módulo de autenticación:

- *Registro o carga de usuarios:* El registro de usuarios se implementó garantizando la seguridad de las credenciales con el uso de los hash. De esta manera las contraseñas

no son almacenadas en texto plano, sino que se producen utilizando el algoritmos bcrypt.

El código define una función que se encarga de registrar un nuevo usuario, en primer lugar consulta la base de datos, haciendo uso del modelo, si ya existe un registro con el mismo nombre lanza un error. Si no existe, la contraseña proporcionada se procesa utilizando el hashContrasena, posteriormente crea una nueva instancia de usuario con los campos usuario, contraseña aseada y un valor booleano de si tiene 2FA. Finalmente guarda el nuevo usuario en la base de datos y retorna un objeto con los datos básicos id, usuario y estado de tiene 2FA.

El uso de hash garantiza que incluso cuando hay acceso no autorizado a la base de datos, las credenciales no son visibles explícitamente y no serán tan fáciles de recuperar o desencriptar.

```
JS autenticacion.servicio.js ×   JS mensajes.servicio.js   JS mensaje.controlador.js   JS mensaje.servicio.js   JS hash.js ...\utilida
sistemaA > src > servicios > JS autenticacion.servicio.js > registrarUsuario
3
4   export const registrarUsuario = async (usuario, contrasena) => {
5
6       const usuarioExistente = await Usuario.findOne({ usuario });
7       if (usuarioExistente) {
8           throw new Error('El nombre de usuario ya está en uso');
9       }
10
11      const contrasenaHasheada = await hashearContrasena(contrasena);
12      const nuevoUsuario = new Usuario({
13          usuario,
14          contrasena: contrasenaHasheada,
15          tiene2FA: false
16      });
17      await nuevoUsuario.save();
18
19      return {
20          id: nuevoUsuario._id,
21          usuario: nuevoUsuario.usuario,
22          tiene2FA: nuevoUsuario.tiene2FA
23      };
24  };
```

Fig. 5. Función registrar usuario del servicio autenticación.

La función para registro de usuario hace uso de una de las utilidades definidas, especificaciones, la utilidad hash.js con su metodo hashearContrasena. Este método o función tiene por objetivo proteger la contraseña antes de almacenarla. Utiliza la librería bcrypt que aplica un algoritmo con un salto para evitar ataques como el uso de tablas precalculadas, al ser 10 aumenta la seguridad haciendo más costoso del descifrado de la contraseña. Recibe la contraseña en texto plano y retorna la versión hasheada.

```
JS hash.js ×   JS jwt.js   <> qr.html   JS otp.js   JS db.js
sistemaA > src > utils > JS hash.js > ...
1   import bcrypt from 'bcrypt';
2   import crypto from 'crypto';
3
4   const RONDAS_SALT = 10;
5
6   export const hashearContrasena = async (contrasena) => {
7       return await bcrypt.hash(contrasena, RONDAS_SALT);
8   };
```

Fig. 6. Función hashearContrasena de la utilidad hash.

- *Inicio de sesión con usuario y contraseña*: El proceso de autenticación con inicio de sesión implementa una función en donde valida las credenciales. Primero busca si el usuario está en la base de datos, si no lo encuentra lanza un error, si si esta, compara la contraseña ingresada con la almacenada (hasheada) haciendo uso de compararContrasena, si esta comparación falla lanza un error, pero si es válida, se retorna un objetivo con los datos básicos del usuario.

```
export const iniciarSesionUsuario = async (usuario, contrasena) => {
    const usuarioEncontrado = await Usuario.findOne({ usuario });

    if (!usuarioEncontrado) {
        throw new Error('Credenciales incorrectas');
    }

    const coincide = await compararContrasena(
        contrasena,
        usuarioEncontrado.contrasena
    );

    if (!coincide) {
        throw new Error('Credenciales incorrectas');
    }

    return {
        id: usuarioEncontrado._id,
        usuario: usuarioEncontrado.usuario,
        tiene2FA: usuarioEncontrado.tiene2FA
    };
};
```

Fig. 7. Función iniciarSesionUsuario del servicio autenticación.

Al igual que la funciona para el registro de usuarios, esta función para el inicio de sesión utiliza la utilidad has.js, pero con la función compararContrasena. Esta permite verificar si una contraseña ingresada coincide con la contraseña almacenada en la base de datos, esta función retorna un valor booleano de acuerdo al resultado coincidencial.

```
export const compararContrasena = async (contrasena, contrasenaHasheada) => {
    return await bcrypt.compare(contrasena, contrasenaHasheada);
};
```

Fig. 8. Función compararContrasena de la utilidad hash.

Adicionalmente, una vez las credenciales son validadas de manera correcta, el sistema genera un token JWT que permite mantener la sesión del usuario de manera segura, esta parte es invocada en el controlador de autenticación. Adicionalmente, es utilizado para autenticar las solicitudes protegidas del sistema,

```
autenticacion.controlador.js ×   mensajes.controlador.js   segundoFactor.controlador.js   autenticacion.middleware.js
istemaA > src > controladores > autenticacion.controlador.js > ...
30  rt const iniciarSesion = async (req, res, next) => {
52
53      const token = generarToken({id: usuarioEncontrado.id, usuario: usuarioEncontrado.usuario});
54
55      res.status(200).json({
56          exito: true,
57          requiere2FA: false,
58          mensaje: 'Inicio de sesión exitoso. Recuerda configurar tu 2FA',
59          token,
60          usuario: usuarioEncontrado
```

Fig. 9. Generación del token en el controlador de autenticación.

Para ello hace uso de una de las utilidades, especificadas, en este caso la utilidad de jwt.js en su función generarToken,recibe un payload con la información del usuario, el id y el usuario, y utiliza el método sign de la librería jwt para crear un token firmado digitalmente.Incluye un secreto que permite no permite que el token sea alterado sin ser detectado y además establece un tiempo de expiración.

```
// Genera un token firmado con los datos del usuario
export const generarToken = (payload) => {
    return jwt.sign(payload, env.JWT_SECRET, {
        expiresIn: env.JWT_EXPIRES_IN
    });
};
```

Fig. 10. Función generarToken de utilidad jwt.js.

Autenticación de dos pasos:

- *Configuracion segundo factor de autenticación:* Para aumentar la seguridad, se crea un servicio exclusivo para el segundo factor (segunfoFactor.servicio,js), este está basado en TOTP, en la configuración inicial se crea un secreto para cada usuario y el código QR que permite al usuario registrar su cuenta en aplicaciones como Google Authenticator.

  Primero, busca al usuario en la base de datos utilizando el Id, si no existe lanza un error, luego, genera un secreto utilizando la función generarSecreto2FA, este secreto se guarda en la base de datos en formato base32. Posteriormente, genera un código QR a partir de la URL de autenticación y finalmente, retorna tanto el código QR como el secreto en base32, permitiendo completar el proceso de configuración del segundo factor de autenticación.

```
export const configurar2FA = async (usuarioId) => {
    const usuario = await Usuario.findById(usuarioId);
    if (!usuario) throw new Error('Usuario no encontrado');

    const secreto = generarSecreto2FA();

    usuario.secreto2FA = secreto.base32;
    await usuario.save();

    const codigoQR = await generarCodigoQR(secreto.urlAutenticacion);

    return {
        codigoQR,
        secretoBase32: secreto.base32
    };
};
```

Fig. 11. Función configurar2FA del servicio segundo factor.

Esta configuración utiliza la otp.js en la carpeta de utilidades con los siguientes métodos. El método generarSecreto2FA que utiliza la librería speakeasy para crear un

secreto asociado a una aplicación de nombres SistemaSeguro_App, lo que permite que los servicios como Google Authenticator reconozcan la cuenta.

```
// Genera un secreto único para el usuario
export const generarSecreto2FA = () => {
    const secreto = speakeasy.generateSecret({
        name: 'SistemaSeguro_App'
    });

    return {
        base32: secreto.base32,
        urlAutenticacion: secreto.otpauth_url
    };
};
```

Fig. 12. Función generarSecreto2FA de la utilidad otp.

El segundo método utilizado es generarCodigoQR que utiliza la librería QRCode para convertir la URL en una imagen codificada en formato Data URL. Retorna una cadena que utiliza el frontend para mostrar el QR y que el usuario lo pueda escanear.

```
// Convierte la URL en un código QR (Base64)
export const generarCodigoQR = async (urlAutenticacion) => {
    try {
        const codigoQR = await QRCode.toDataURL(urlAutenticacion);
        return codigoQR;
    } catch (error) {
        console.error('Error generando el código QR:', error);
        throw new Error('No se pudo generar el código QR');
    }
};
```

Fig. 13. Función generarCodigoQR de la utilidad otp.

- *Verificación y activación de segundo factor*: Una vez lista la configuración, se procede con la verificación y activación de segundo factor, en donde el usuario debe ingresar el código valido generado por su aplicación autenticadora. Esta función busca al usuario por su ID, valida el código 2FA ingresado usando el secreto almacenado, si el código es correcto, activa el 2FA cambiando la propiedad tiene2FA a true y guarda el cambio en la base de datos.

```
export const verificarYActivar2FA = async (usuarioId, token) => {
    const usuario = await Usuario.findById(usuarioId);
    if (!usuario) throw new Error('Usuario no encontrado');

    const esValido = verificarToken2FA(usuario.secreto2FA, token);

    if (!esValido) {
        throw new Error('El código 2FA es incorrecto o ha expirado');
    }

    usuario.tiene2FA = true;
    await usuario.save();

    return true;
};
```

Fig. 14. Función verificarYActivar2FA del servicio segundo factor.

- *Validación de login segundo factor:* Por último, se realiza la validación durante el inicio de sesión, es decir, durante el proceso de inicio de sesión, si el usuario tiene activado el 2FA se solicita un código adicional antes de conceder el acceso. De esta manera se garantiza que el acceso solo sea permitido si ambos factores, tanto la contraseña como el código temporal son correctos. Esta función busca al usuario, pero además de comprobar que el 2FA ya está habilitado, dispuesta valida el código 2FA ingresado, es correcto, permite continuar con el proceso de autenticación.

```
export const validarLogin2FA = async (usuarioId, token) => {
    const usuario = await Usuario.findById(usuarioId);
    if (!usuario) throw new Error('Usuario no encontrado');

    if (!usuario.tiene2FA) {
        throw new Error('El 2FA no está habilitado para este usuario');
    }

    const esValido = verificarToken2FA(usuario.secreto2FA, token);
    if (!esValido) {
        throw new Error('Código 2FA incorrecto');
    }

    return usuario;
};
```

Fig. 14. Función validarLogin2FA del servicio segundo factor.

Las últimos dos funciones, utilizan otp.js en la función verificarToken2FA que se encarga de verificar si un código de 2FA es válido, utiliza la librería speakeasy para comparar el token ingresado por el usuario con el código generado a partir del formato base32.

```
// Verifica el código ingresado por el usuario
export const verificarToken2FA = (secretoBase32, tokenUsuario) => {
    return speakeasy.totp.verify({
        secret: secretoBase32,
        encoding: 'base32',
        token: tokenUsuario,
        window: 1 // tolerancia de tiempo
    });
};
```

Fig. 15. Función verificarToken2FA de la utilidad otp.

De esta manera se completa la validación de credenciales, se solicita el segundo factor y se da acceso solo si ambos factores son correctos, con un token JWT final que permite validar las operaciones posteriores de forma segura.

Middleware de autenticación:

Con el fin de proteger las rutas del sistema, se realizó la implementación de un middleware que valida el token JWT en cada solicitud. Este es un mecanismo que permite identificar al usuario autenticado sin necesidad de reenviar las credenciales en cada petición.

```
18      //Veririca el token
19      const payloadDecodificado = verificarToken(token);
20
21      if (!payloadDecodificado) {
22          return res.status(401).json({
23              exito: false,
24              mensaje: 'Token inválido o expirado. Por favor, inicia sesión nuevamente.'
25          });
26      }
27
28      // Guarda los datos del usuario en req
29      req.usuario = payloadDecodificado;
30      next();
31  };
```

Fig. 16. Middleware para autenticación.

Hace uso de la utilidad jwt.js para verificar si el token es válido y no ha expirado, en caso de que se así se muestra un error, si no lo es se deja seguir con la operación en cuestión.

```
// Verifica si un token es válido y no ha expirado
export const verificarToken = (token) => {
    try {
        return jwt.verify(token, env.JWT_SECRET);
    } catch (error) {
        return null;
    }
};
```

Fig. 17. Función verificarToken de la utilidad jwt.

Manejo de errores:

Se implementó un middleware global que permite la gestión de manera centralizada de errores, este componente lo que hace es capturar las excepciones y garantizar respuestas consistencia antes distintos fallos del sistema.

```
sistemaA > src > middlewares > JS error.middleware.js > ...
1   export const manejadorErrores = (err, req, res, next) => {
2       console.error(`[Error]: ${err.message}`);
3       if (err.stack) {
4           console.error(err.stack);
5       }
6
7       const codigoEstado = res.statusCode && res.statusCode !== 200 ? res.statusCode : 500;
8
9       res.status(codigoEstado).json({
10          exito: false,
11          mensaje: err.message || 'Error interno del servidor',
12          stack: process.env.NODE_ENV === 'production' ? null : err.stack
13      });
14  };
```

Fig. 16. Middleware para errores.

Comunicación entre sistemas:

La comunicación entre sistema inicia con el servicio de manejo dispuesto en el sistema A (mensaje.servicio.js) permite el envío de mensaje hacia el sistema B incluyendo la información más relevante como el contenido, el remitente y la marca temporal.

Implementación de sistema A que permite la comunicación:

El servicio de mensaje implementado en el Sistema A es el encargado de enviar mensajes de manera segura de este mismo hasta el otro, denominado Sistema B. Primero, construye una carga de datos en donde se incluye el contenido del mensaje, el usuario remitente y la fecha o marca de tiempo, esto permite identificar cuando fue enviado el mensajes. Luego, se genera un hash o generación de huella del mensaje utilizando la función generarHashMensaje, esto con el objetivo de garantizar la integridad de la información, en otras palabras, este hash garantiza que el mensaje no haya sido modificado durante la transmisión.

Seguidamente, se realiza la petición HTTP de tipo POST al sistema B mediante fetch, enviando la carga y el has en formato JSON y almacena la respuesta en la variable datos.

sistemaA > src > servicios > JS mensajes.servicio.js > ...

```
1   import { generarHashMensaje } from '../utils/hash.js';
2
3   // Servicio para enviar el mensaje al Sistema B
4   export const enviarMensajeASistemaB = async (contenido, usuario) => {
5
6       const carga = {
7           contenido: contenido,
8           remitente: usuario,
9           timestamp: new Date().toISOString()
10      };
11
12      const hashMensaje = generarHashMensaje(carga);
13      const urlSistemaB = process.env.SISTEMA_B_URL || 'http://127.0.0.1:4000/api/mensajes/recibir';
14
15      try {
16          const respuesta = await fetch(urlSistemaB, {
17              method: 'POST',
18              headers: {
19                  'Content-Type': 'application/json'
20              },
21              body: JSON.stringify({
22                  carga: carga,
23                  hash: hashMensaje
24              })
25          });
26
27          const datos = await respuesta.json();
```

Fig. 17. Servicio de envío de mensajes parte uno.

Siguiendo con el código de este servicio, se valida si la respuesta fue exitosa, si no lo fue se envía un error pero si sí lo fue, retorna un objeto indicando el éxito del envío, la respuesta del sistema B y el hash enviado.

```
        const datos = await respuesta.json();

        if (!respuesta.ok) {
            throw new Error(
                datos.mensaje || `El Sistema B rechazó el mensaje (Status: ${respuesta.status})`
            );
        }

        return {
            exito: true,
            respuestaSistemaB: datos,
            hashEnviado: hashMensaje
        };

    } catch (error) {
        console.error('Error de conexión con Sistema B:', error.message);
        throw new Error('No se pudo entregar el mensaje al Sistema B. ¿Está activo?');
    }
};
```

Fig. 18. Servicio de envío de mensajes parte dos.

Este código utiliza una función externa denominada generarHashMensaje de la utilidad hash.js, esta función convierte la información o carga recibida en una cadena de texto utilizando JSON.stringify. Luego, aplica el algoritmo SHA-256 con crypto para producir un valor en hexadecimal que representa el contenido, cualquier cambio en el mensaje genera un hash diferente, por lo cual esta parte es fundamental para mantener la integridad.

```
export const generarHashMensaje = (cargaMensaje) => {
    const cadenaDatos = typeof cargaMensaje === 'string'
        ? cargaMensaje
        : JSON.stringify(cargaMensaje);

    // hash SHA-256
    return crypto
        .createHash('sha256')
        .update(cadenaDatos)
        .digest('hex');
};
```

Fig. 19. Función generarHashMnesaje de la utilidad hash.

Implementación de sistema B que permite la comunicación:

El Sistema B actúa como un servicio de recepción. No gestiona usuarios, sino que únicamente se encarga de recibir la información enviada por parte del Sistema A y verifica que los datos no hayan sido modificados en la red durante el trayecto entre ambos sistemas.

El módulo de integridad de mensajes, se encarga de:

- Verificación de integridad: La recepción de los mensajes se manejan a través del controlador (mensaje.controlador.js), el cual intercepta la petición HTTP y extrae el Payload (Contenido del mensaje en texto claro), como también el hash firmado por el Sistema A. Para garantizar la integridad de los datos, el

sistema B no se confía solo del hash recibido, sino que este vuelve a recalcular el hash a partir de los datos recibidos mediante una copia su propio algoritmo HASH (utilidades/hash.js) del Sistema A.

- Rechazo de mensajes alterados: La base del sistema B se encuentra en la comparación de los hash creados en cada uno de los sistemas por separado. Si ambos hash coinciden el sistema cataloga el mensaje como 'íntegro' y se autoriza su persistencia en la base de datos de MongoDB. Por lo contrario, si el sistema llega a detectar una diferencia mínima entre los dos sistemas, el sistema asume que la integridad fue comprometida y rechaza la operación devolviendo un estado HTTP 404 e impidiendo que la información se almacene en la base de datos.

```
import Mensaje from '../modelos/mensaje.modelo.js';
import { generarHashMensaje } from '../utilidades/hash.js';

// Recibe un mensaje, verifica su integridad y lo guarda si es auténtico
export const recibirVerificarMensaje = async (carga, hashRecibido) => {
    const hashCalculado = generarHashMensaje(carga);

    const esIntegro = hashCalculado === hashRecibido;

    if (!esIntegro) {
        console.error('ALERTA DE SEGURIDAD: Se ha detectado un mensaje alterado en tránsito.');
        throw new Error('Integridad comprometida: El mensaje ha sido alterado y fue rechazado.');
    }

    const nuevoMensaje = new Mensaje({
        contenido: carga.contenido,
        remitente: carga.remitente,
        enviadoEl: carga.timestamp,
        hashRecibido: hashRecibido,
        estadoIntegridad: 'Íntegro'
    });

    await nuevoMensaje.save();

    return nuevoMensaje;
};
```

Fig. 20. Servicio de recepción de mensajes.

El servicio recibe como parámetros la carga (mensaje) y el hash generado en el Sistema A, hace uso de la función 'generarHashMensaje' que es exactamente igual a la que tiene el sistema emisor y calcula el hash. Luego, verifica si el hash calculado es exactamente idéntico al hash recibido, si detecta una mínima diferencia lanza una excepción indicando que la integridad del mensaje ha sido comprometida.

Únicamente, cuando el hash no ha sido modificado, se mapean los datos en base al modelo, asociando el 'timestamp' enviado por el Sistema A a el campo 'enviadoEl' y se asigna de manera inmutable la etiqueta de "estadoIntegridad: 'integro' ". Finalmente, se guarda el mensaje de forma permanente en la base de datos del sistema.

3. Evidencias

Se realizaron pruebas principalmente con Postman, estas permitieron verificar el funcionamiento y cumplimiento de los requisitos. Las pruebas ejecutadas se muestran a continuación.

1. Para el módulo de autenticación se realizó una primera prueba que permite el registro de usuarios, con un 200 de respuesta.

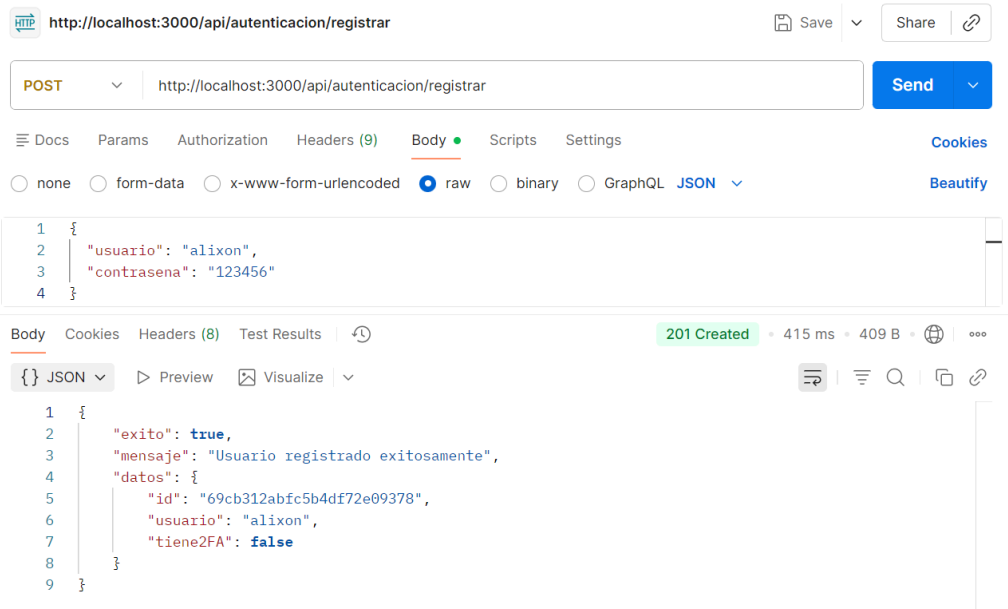

Fig. 21. Prueba Postman registro de usuario.

2. Se intentó el ingreso al sistema con credenciales incorrectas donde el resultado fue 401 no autorizado, y denegado el acceso al sistema.

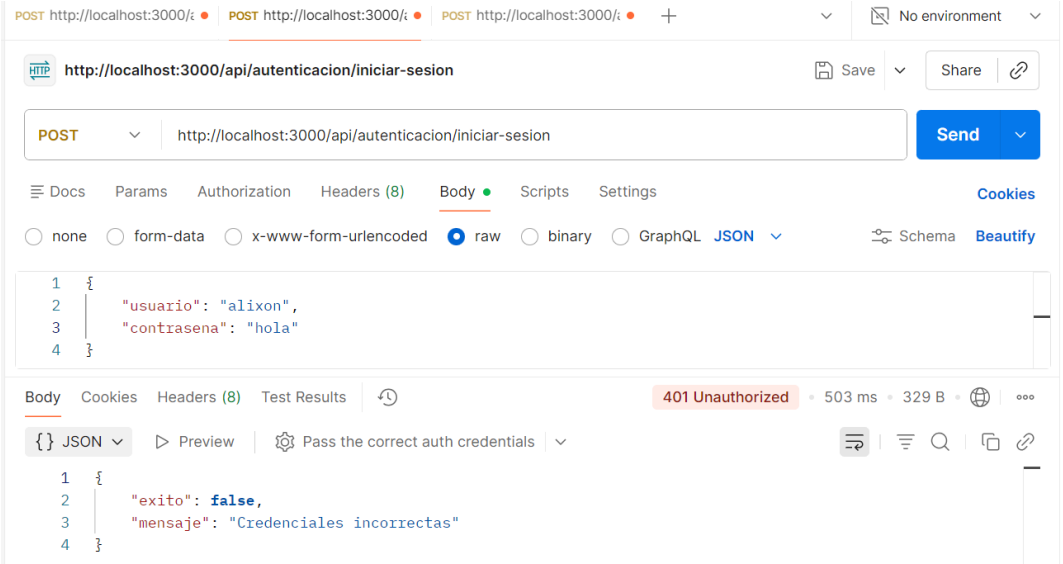

Fig. 22. Prueba Postman iniciar sesión con credenciales incorrectas.

3. Se realizó la prueba respectiva para el inicio de sesión, únicamente con credenciales correctas, como se puede evidenciar en la imagen, se observa que el campo requiere2FA está en false, puesto que para este usuario no se ha verificado este segundo factor. Además, la respuesta genera el token respectivo y el id del usuario que serán útiles para las próximas pruebas.

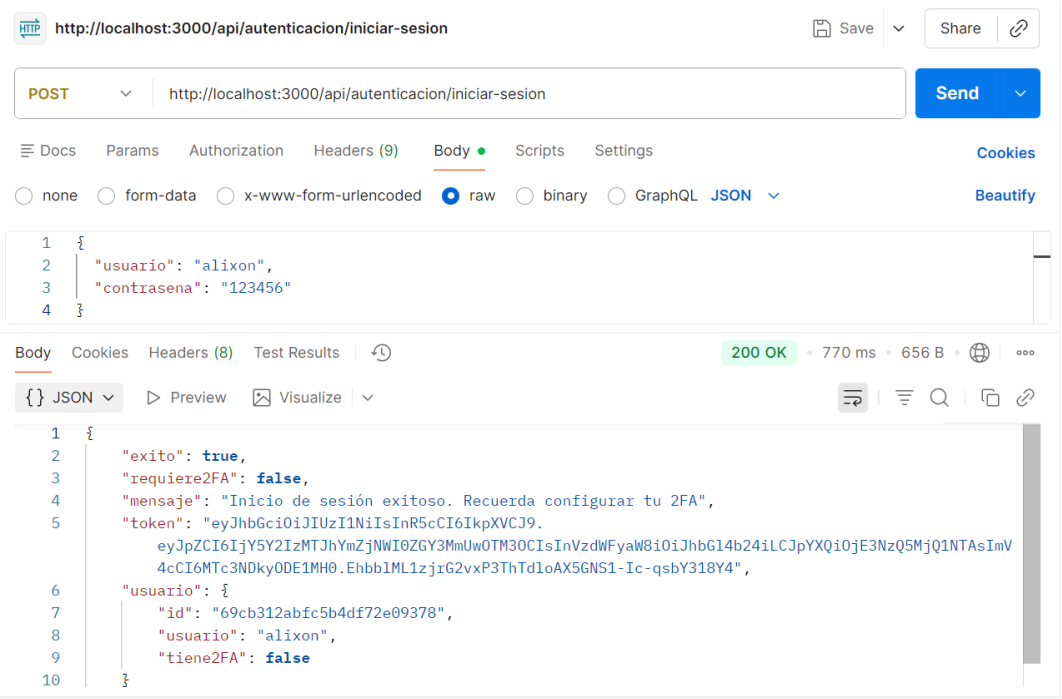

Fig. 23. Prueba Postman iniciar sesión con credenciales correctas.

Es importante guardar el token, pues este se pide como requisito en el Bearer en las próximas operaciones.
Token:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5Y2IzMTJhYmZjNWI0ZGY3MmUwOTM3OCIsInVzdWFyaW8iOiJhbGl4b24iLCJpYXQiOjE3NzQ5MjQ1NTAsImV4cCI6MTc3NDkyODE1MH0.EhbblML1zjrG2vxP3ThTdloAX5GNS1-Ic-qsbY318Y4

4. Se realizó una tercera prueba para la generación del segundo factor que incluye la creación del secreto y el código QR. Para esta operación, se requiere el Token generado en el paso anterior, tal como se muestra en la Fig. #.

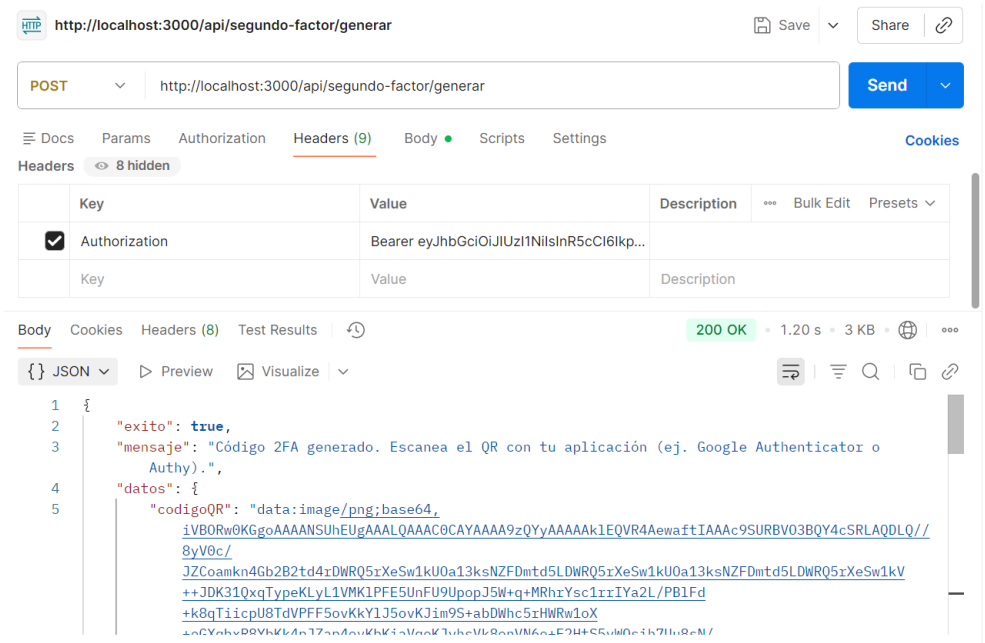

Fig. 24. Prueba Postman para generación de QR

La respuesta da un codigoQR que contiene la imagen generada. Este se procede a leer con un html exclusivo para esta operación. El HTML planteado para eso se muestra en la Fig. #:

```
JS jwt.js   <> qr.html U ×   JS otp.js   JS db.js   JS env.js   JS autenticacion.servicio.js
<> qr.html > html > body > script
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="UTF-8">
5    <title>Mostrar imagen Base64</title>
6  </head>
7  <body>
8
9  <script>
10 const base64 = "iVBORw0KGgoAAAANSUhEUgAAALQAAAC0CAYAAAA9zQYyAAAAAklEQVR4AewaftIAAAc9SURBVO3BQY4
11
12 const img = document.createElement("img");
13 img.src = "" + base64;
14
15 document.body.appendChild(img);
16 </script>
17
18 </body>
19 </html>
```

Fig. 25. Código HTML para leer QR.

Una vez se abre, el resultado se logra visualizar, en este paso se procede a utilizar la página o aplicación autenticadora que brinde el token o el código que permite pasar este filtro de segundo factor de autenticación.

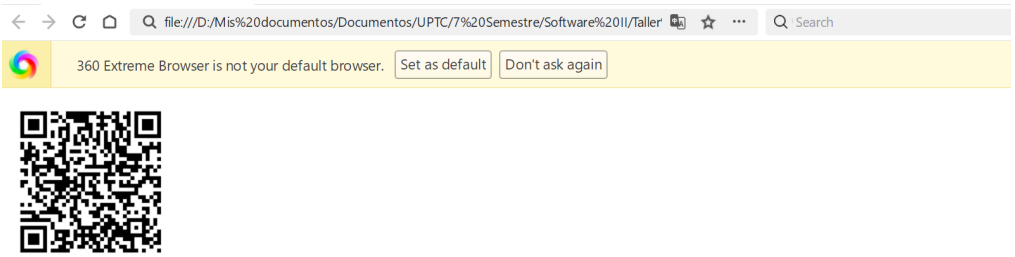

Fig. 26. Evidencia de QR generado.

En este caso la aplicación del Google Authenticator da el token 601814, así que procede a verificar en Postman obteniendo una respuesta de 200 ok.

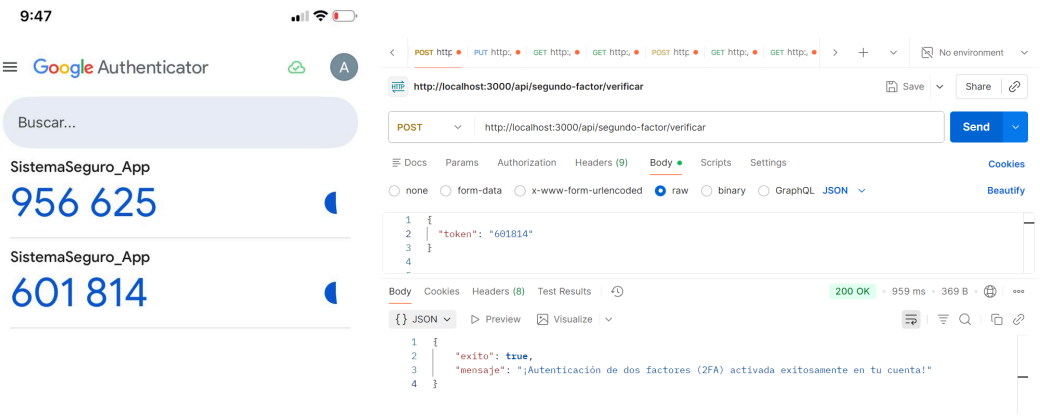

Fig. 27. Prueba Postman para verificación de 2FA.

De igual manera se comprobó el funcionamiento del sistema cuando el token o código brindado por la aplicación es erróneo o está expirado, en este caso se obtiene un 400.

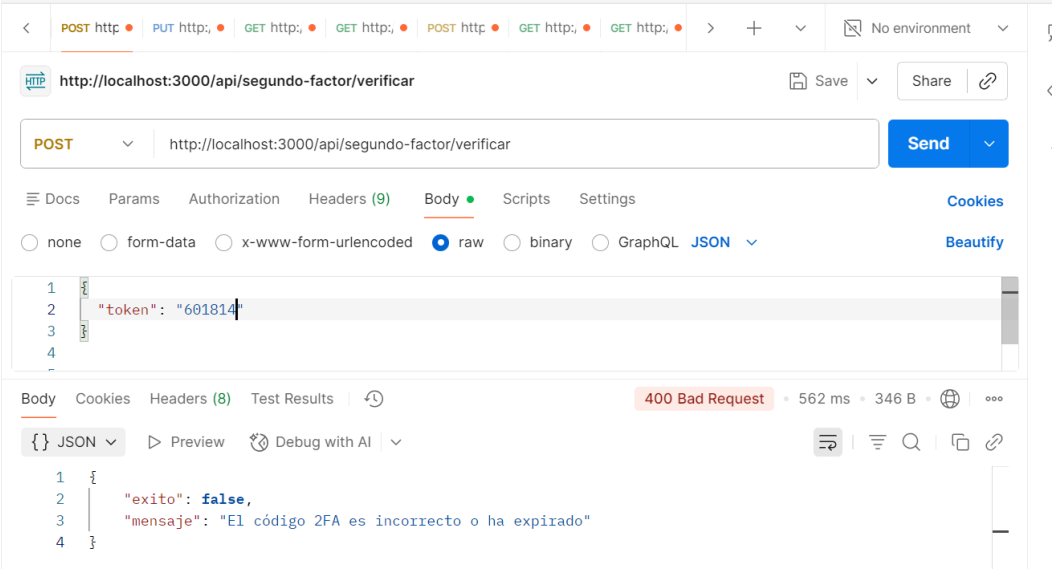

Fig. 28. Prueba Postman con token incorrecto en verificación de 2FA.

5. Al volver a inciar sesion con el mismo usuario que ya verificó una primera vez el segundo factor de autenticación, el campo de requiere2FA se modifica a true

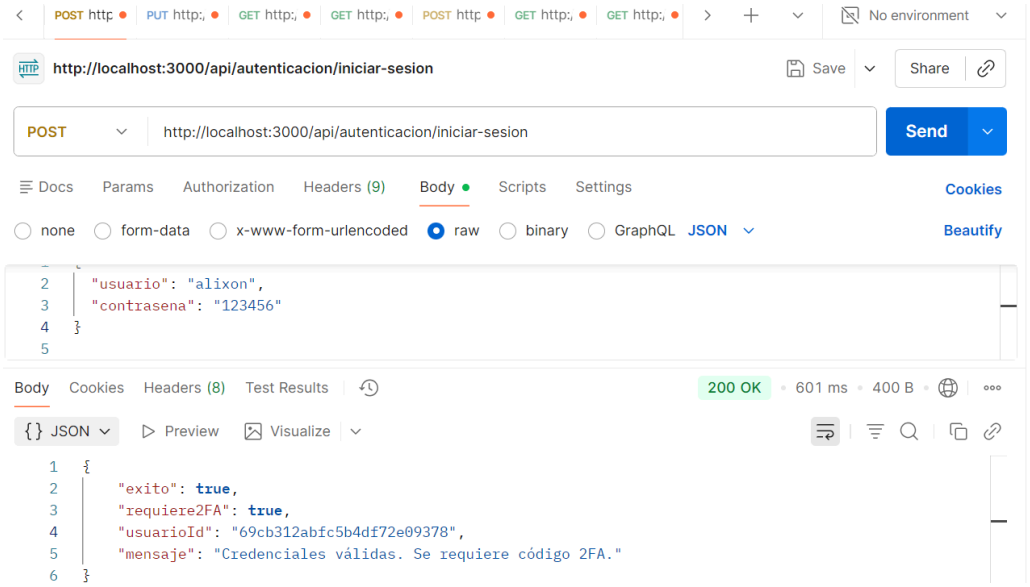

Fig. 29. Prueba Postman cambio de campo requiere2FA cuando el usuario ya lo ha verificado.

De esta manera ya no se utiliza la ruta verificar, sino validar-login en donde se deben enviar el Id del usuario y el token actualizado, esto da como respuesta 200 ok y un token final que es el que permite enviar mensajes.

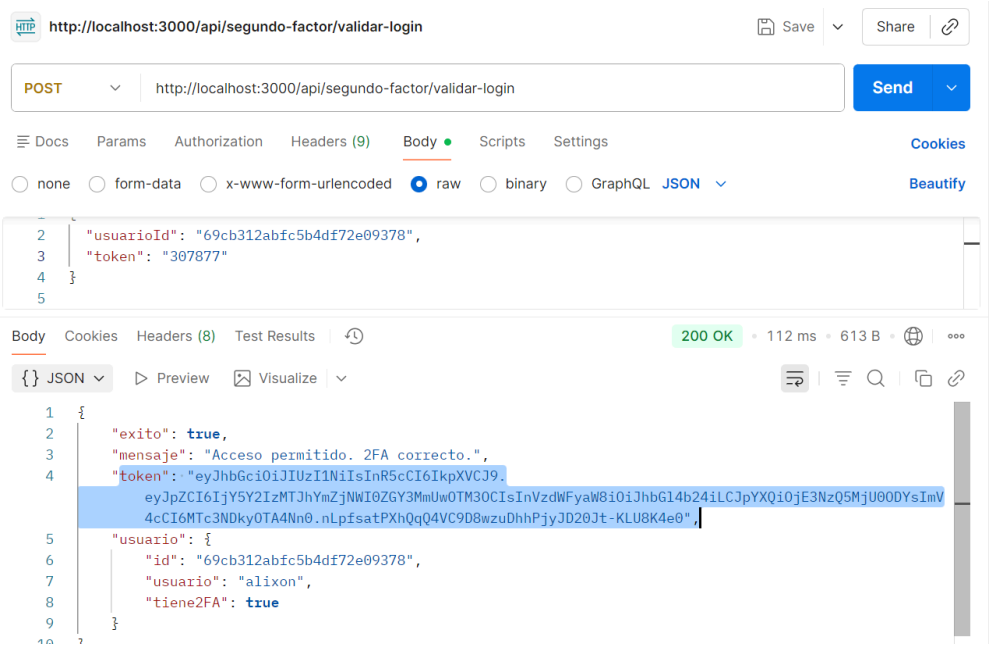

Fig. 30. Prueba Postman validar login con 2FA ya verificado.

Es el token definitivo que permite enviar mensajes
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5Y2IzMTJhYmZjNWI0ZGY3MmUwOTM3OCIsInVzdWFyaW8iOiJhbGl4b24iLCJpYXQiOjE3NzQ5MjU0ODYsImV4cCI6MTc3NDkyOTA4Nn0.nLpfsatPXhQqQ4VC9D8wzuDhhPjyJD20Jt-KLU8K4e0

6. Las pruebas de envío de mensaje se realizan únicamente cuando el sistema B está activo o corriendo en su respectivo puesto, además se debe utilizar el token final en el Bearer y en el Body enviar el contenido del mensaje. Tal como se muestra en la figura #31 con una respuesta 200 ok.

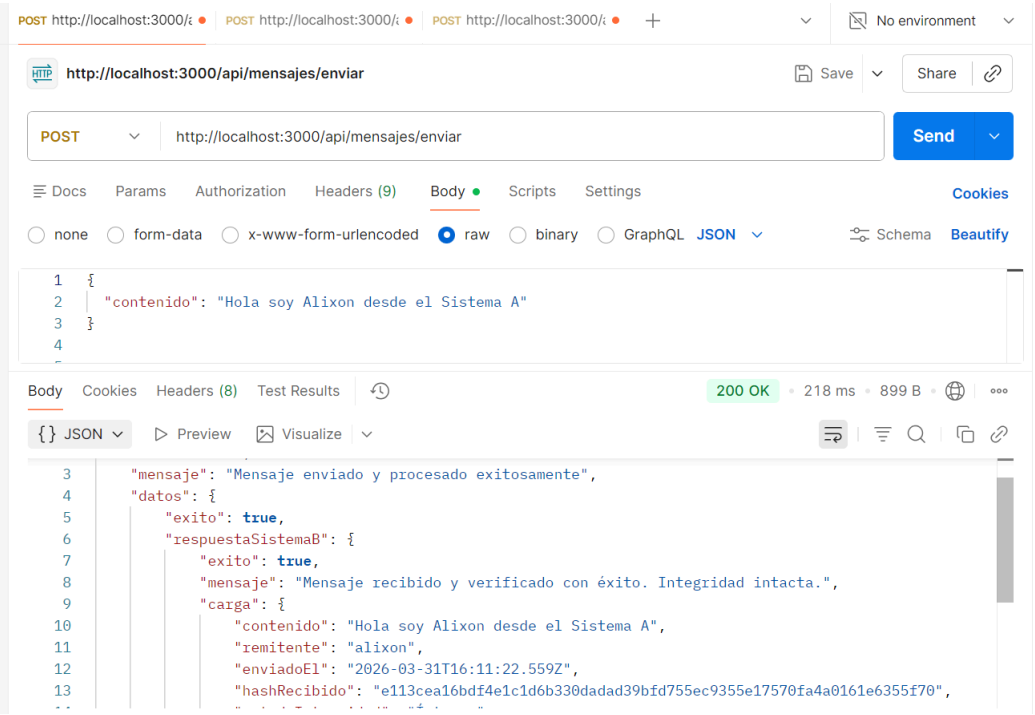

Fig. 31. Prueba Postman envío de mensaje a Sistema B.

7. Para la prueba de mensaje alterado, se hace una petición POST al Sistema B en donde se recibe el contenido del mensaje pero el Hash se modifica por uno que no corresponde al que calculo y envio el Sistema A. Al hacer esta prueba se visualiza que el mensaje recibido fue comprometido, devolviendo un código de respuesta 403 Fofbidden.

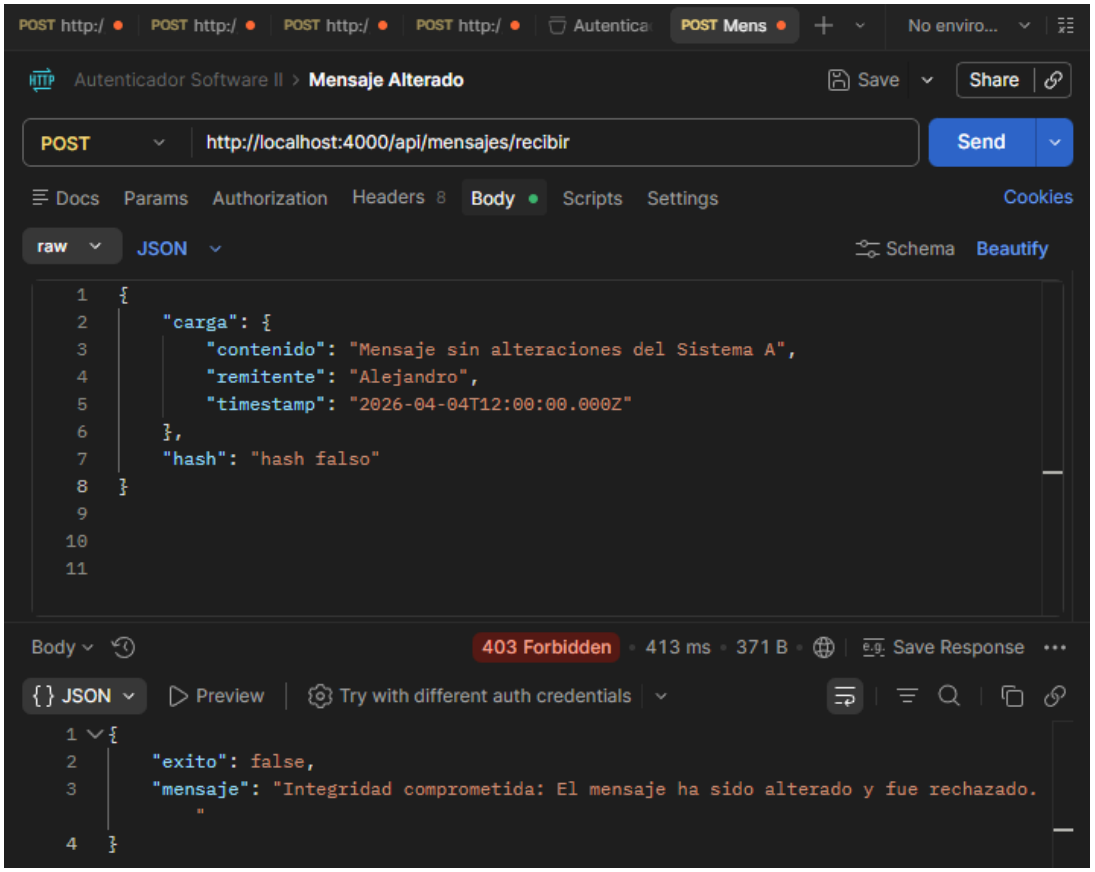

Fig. 32. Prueba Postman mensaje alterado.

Pruebas con Frontend.

Después de la pruebas de desarrollo, se realizaron las pruebas del lado del cliente (Frontend)

1. La primera prueba realizada, permite verificar el registro de usuarios. Para lo cual, el usuario debe de ingresar el nombre y una contraseña.

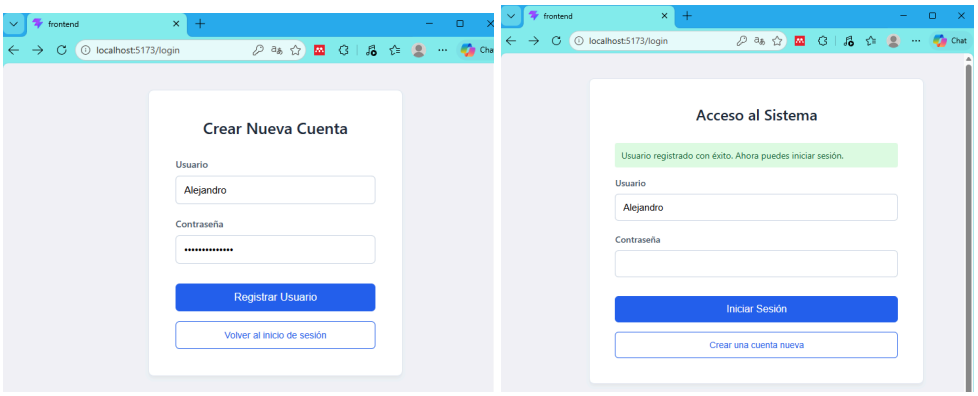

Fig. 34. Prueba Fontend registro de usuario.

Una vez registrado, el sistema lo redirige al login donde el usuario debe de ingresar la contraseña

2. En la segunda prueba se verifica que cuando el usuario ingrese contraseñas incorrectas, el sistema no lo deje ingresar

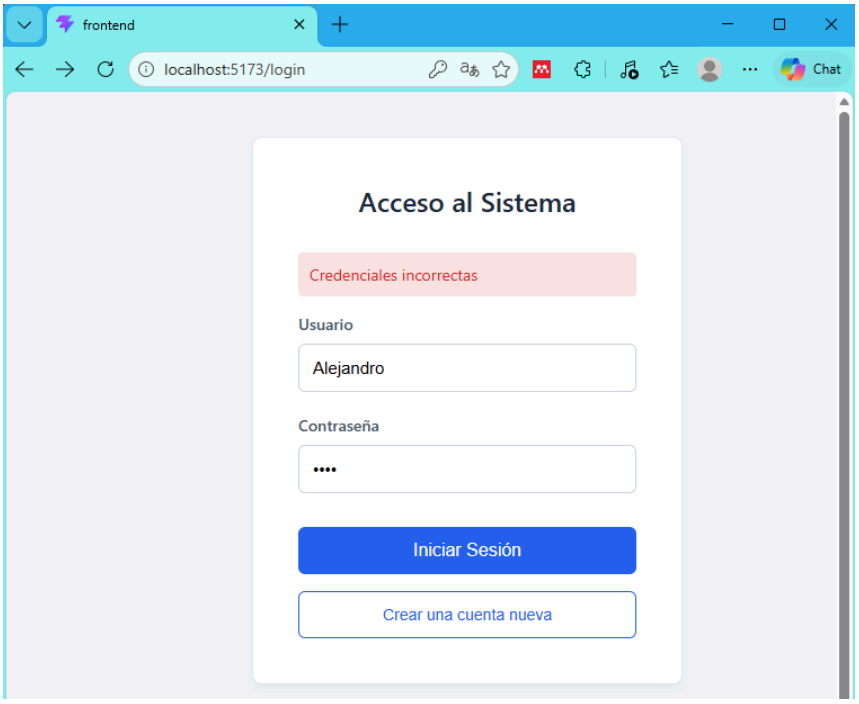

Fig. 35. Prueba Frontend de credenciales incorrectas.

3. La tercera prueba consiste en verificar que una vez que el usuario ingresa al sistema por primera vez, el sistema automáticamente le pida la activación del segundo factor de autenticación:

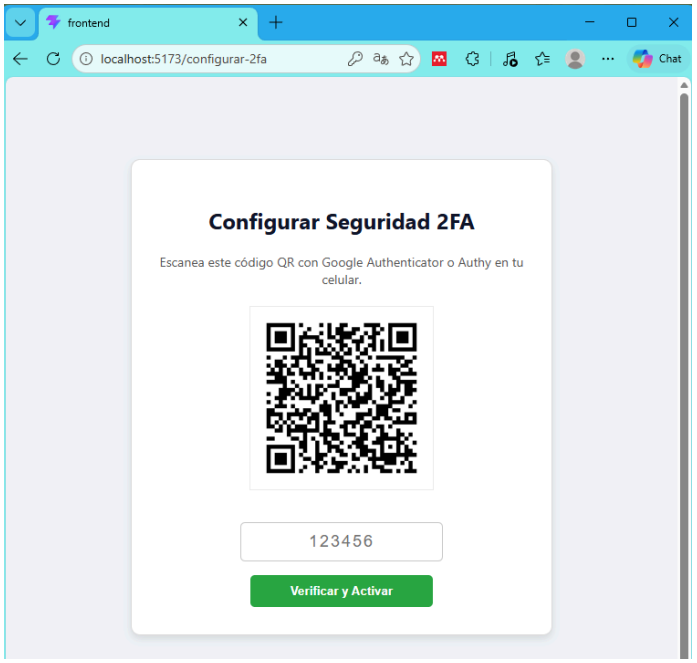

Fig. 36. Prueba Frontend de creación QR.

4. Cuarta prueba, el usuario ingresa un código incorrecto o expirado en el 2FA.

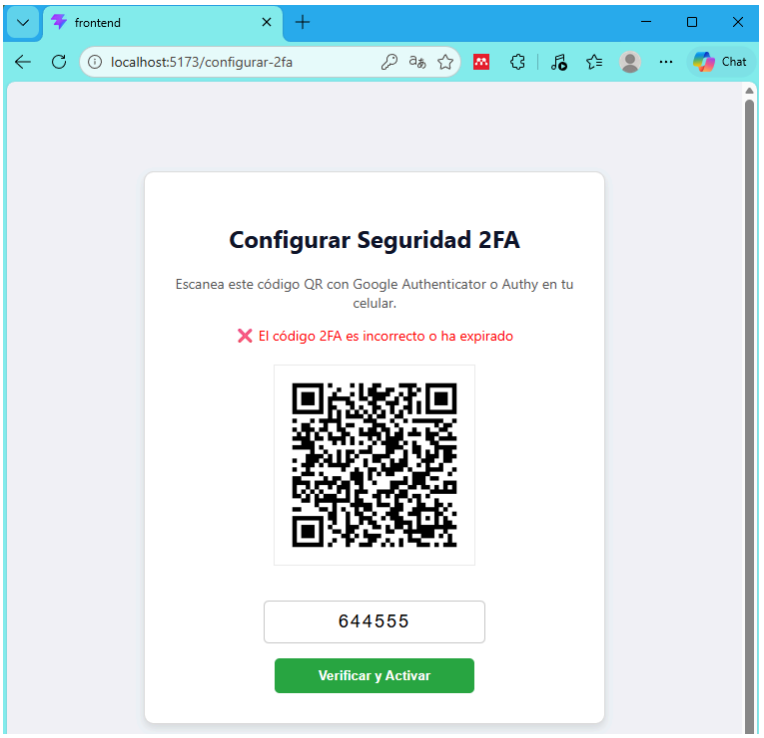

Fig. 37. Prueba Frontend 2FA incorrecto.

Cuando el código es correcto el sistema permite el ingreso del usuario y lo redirige a la página principal para el envío de mensajes, tal y como se muestra en la siguiente prueba.

5. En la última prueba, se verifica el envío de mensaje entre sistema y la integridad de los datos transmitidos, mediante la comparación de los Hash del sistema de origen y del sistema que recibe el mensaje

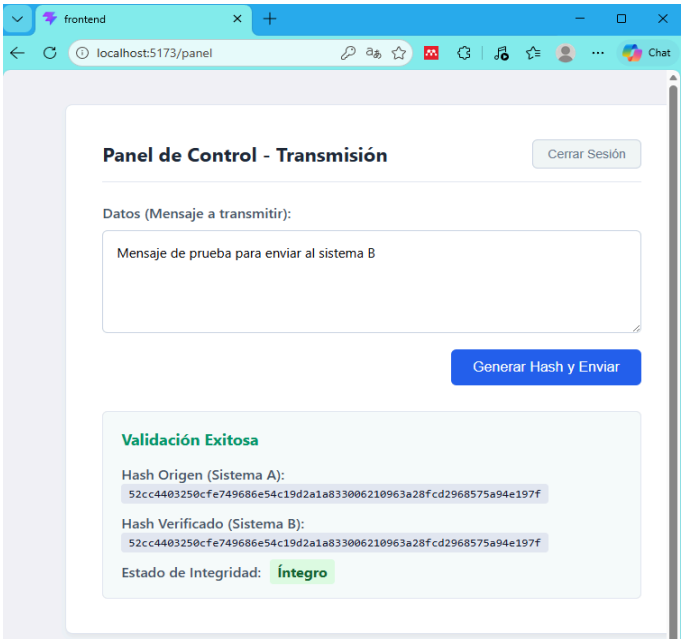

Fig. 38. Prueba Frontend envío e integridad de mensajes.

PRUEBAS TIPO CHECKLIST

VALOR	REQUISITO	ESTADO
Autenticación	Registro o carga de usuarios	**APROBADO**
Autenticación	Inicio de sesión con usuario y contraseña	**APROBADO**
Autenticación	Segundo factor de autenticación	**APROBADO**
Comunicación	Envío de mensajes entre sistemas	**APROBADO**
Comunicación	Verificación de integridad	**APROBADO**
Comunicación	Rechazo de mensajes alterados	**APROBADO**
Autenticación de dos paso	Validación de credenciales	**APROBADO**
Autenticación de dos paso	Solicitud de segundo factor	**APROBADO**
Autenticación de dos paso	Acceso solo si ambos factores son correctos	**APROBADO**
Integridad de mensajes	Generación de la huella del mensaje (hash)	**APROBADO**
Integridad de mensajes	Verificación en el sistema receptor	**APROBADO**
Integridad de mensajes	Rechazo si el mensaje fue alterado	**APROBADO**
Escenarios obligatorios	Credenciales y 2FA correctos → acceso permitido	**APROBADO**
Escenarios obligatorios	Credenciales incorrectas → acceso denegado	**APROBADO**
Escenarios obligatorios	2FA incorrecto → acceso denegado	**APROBADO**

Integridad	Mensaje íntegro → aceptado	**APROBADO**
Integridad	Mensaje alterado → rechazado	**APROBADO**

4. Análisis.

El sistema implementado que incorpora la autenticación en dos pasos y verifica la integridad del mensaje enviado entre dos sistemas, fue desarrollado con un enfoque modular que se basa en la separación por capas, tal como se mostró en la primera sección de este documento. Esta organización, permite la separación de responsabilidades como la autenticación, el segundo factor, la verificación de integridad y la comunicación entre sistemas. Es importante mencionar que de esta manera se mantiene un diseño limpio y escalable, además de facilitar la reutilización de componentes y la modificabilidad en partes específicas del sistema sin afectar el resto de él.

Dentro de las principales ventajas de la solución implementada, se encuentra la alta seguridad en la autenticación de inicio de sesión con la implementación de doble facto (2FA) lo que reduce el riesgo de acceso no autorizados, incluso si la contraseña es descubierta por un atacante este necesito el segundo factor, la garantía de integridad de los mensajes con el uso de funciones hash, especificacmente SHA-256 que permite detectar cualquier modificación, la separación de responsabilidades y el cumplimiento de los escenarios obligatorios, validados correctamente en las evidencias.

Por otro lado, como desventajas y trade-off teniendo en cuenta que el atributo de priorización es la seguridad, se generan cierto sacrificios, entre ellos: mayor complejidad del sistema, principalmente por la separación en varios módulos y la implementación de mecanismo de seguridad que hacen más complejo el desarrollo, además impacto en el atributo de rendimiento, por el cálculo de hashes y la validación del 2FA que llevan a pequeños retrasos en el procedimiento y la disminución de la usabilidad por el proceso de inicio de sesión más lento y menos cómodo para el usuario.

De manera general, se puede decir que los atributos de calidad sacrificados al priorizar la seguridad son la usabilidad, el rendimiento y la simplicidad.

5. Trabajo Futuro

La solución implementada cumple con los requisitos planteados en el documento guía, priorizando la seguridad con el uso de la autenticación de dos pasos y verificación de integridad de mensajes. Siendo un sistema base sólido para un trabajo futuro o modificaciones de mejora en diferentes ámbitos, como el de la mensajería instantánea.

La implementación del factor de autenticación en dos pasos mejoró la seguridad del intercambio de mensajes entre sistemas, pero por sí sola aún tiene riesgos de que un atacante intercepte la red y logre modificar simultáneamente el contenido del mensaje y el hash enviados, para engañar la verificación de integridad del sistema receptor. Por lo tanto, se sugiere como mejora futura, desarrollar e integrar al sistema de autenticación en dos pasos actual, un sistema de firmas digitales asimétricas (RSA o ECC) o un HMAC (Código de autenticación basado en Hash), que incluya una clave secreta compartida entre el Sistema A y el Sistema B. De esta manera aunque el atacante logre cambiar el mensaje y el hash, no podrá

pasar la verificación de integridad del mensaje en el sistema receptor porque la clave secreta no viaja por la red, sino que solamente estará disponible en los sistemas de origen y destino.